Tech Notes: Day 1 Core Activities

1.1 Parameterized ALU

- **Design Approach:** Implemented a 32-bit parameterized ALU using a combinational always block with a case statement for opcode decoding. To accurately calculate the carry and overflow flags for subtraction and SLT (Set Less Than), I routed the b operand through a 2's complement multiplexer and used a 33-bit internal sum_res wire to capture the carry-out bit.
- **Verification Steps:** Built a self-checking testbench using a check_output task. Forced 25 directed test vectors, specifically targeting corner cases like maximum positive/negative bounds to trigger overflow, and mixed-sign comparisons for the SLT instruction.
- **Results & Issues:** The design passed all 25 vectors with 0 errors. A minor initial issue with the arithmetic shift (SRA) sign extension was resolved by ensuring the operands were declared as signed in the testbench.

1.2 Up/Down Counter

- **Design Approach:** Designed an 8-bit counter using a single synchronous always @(posedge clk) block. Implemented strict control signal priority: synchronous rst overrides load, which overrides en (enable). The up_down signal dictates the arithmetic direction when enabled.
- **Verification Steps:** Verified sequential transitions by asserting reset, performing a parallel load of 8'hAA, counting up past the loaded value, toggling the direction to count down, and finally testing the enable hold state.
- **Results & Issues:** Simulation passed all state checks. Ensured that the synchronous reset properly required a clock edge to evaluate, distinguishing it from an asynchronous reset implementation.

1.3 Barrel Shifter

- **Design Approach:** Built a 32-bit single-cycle shifter relying on standard Verilog shift operators (<<, >>, >>>). Used a 2-bit shift_type control signal to multiplex between Logical Left (SLL), Logical Right (SRL), and Arithmetic Right (SRA). The input was explicitly declared as signed to ensure the >>> operator duplicated the sign bit correctly for SRA.
- **Verification Steps:** Automated the testbench using a for loop to sweep the shift amount (shamt) from 0 to 31 for all three shift types in a single simulation run, comparing the output against a behavioral reference model.

- **Results & Issues:** Passed all 96 shift combinations. No major issues, though care had to be taken to use $unsigned() in the testbench for the SRL reference model to prevent unintended sign extension during verification.

1.4 Register File

- **Design Approach:** Created a 32x32 memory array using reg [31:0] registers [0:31]. Implemented synchronous writes and asynchronous reads. To handle the read-after-write (RAW) hazard in the same clock cycle, I added a continuous assignment bypass multiplexer: if we is high and the read address matches the write address, the output forwards the incoming wd (write data) directly.
- **Verification Steps:** Applied directed tests to ensure register x0 remains strictly hardwired to 0 even if a write is attempted. Tested the internal data forwarding (bypass logic) by reading and writing to register 5 simultaneously. Followed up with a loop of random read/write operations.
- **Results & Issues:** Passed all directed assertions. The bypass logic correctly resolved the same-cycle RAW hazard without requiring a pipeline stall.

Tech Notes: Day 1 Additional Activities

1.5 Ripple-Carry Adder/Subtractor

- **Design Approach:** Used a generate block to instantiate 8 full_adder modules. The mode bit handles subtraction via a 2's complement approach (XORing the B operand and injecting mode as the initial carry-in).
- **Verification Steps:** Created an exhaustive nested for loop in the testbench traversing all 65,536 possible combinations of A and B for both addition and subtraction. Validated outputs against behavioral operators (+ and -).
- **Results & Issues:** All tests passed. Correctly predicting the signed overflow flag in the behavioral testbench required careful boolean logic masking to match the RCA's internal carry_out[7] ^ carry_out[8].

1.6 Priority Encoder and Decoder

- **Design Approach:** Implemented the 8-to-3 priority encoder in two distinct styles: a casez block mapping don't-cares (?), and an if-else chain prioritizing the MSB down to the LSB. The decoder was a standard left-shift operation masked by an enable bit.
- **Verification Steps:** Instantiated both encoder versions in the testbench simultaneously. Swept inputs from 0 to 255, asserting that both out and valid signals matched perfectly every cycle.

- **Results & Issues:** The testbench confirmed that the casez and if-else architectures simulate identically with 0 errors.

1.7 LFSR and Shift Register

- **Design Approach:** Designed the LFSR using a synchronous process and an XOR feedback network (x^8 + x^6 + x^5 + x^4 + 1) to ensure maximal sequence length. Designed a separate universal shift register supporting shift-left, shift-right, and parallel load using a case statement.
- **Verification Steps:** The LFSR testbench loaded a seed, recorded the initial output, and counted clock cycles until the initial value reappeared. It also included an assertion to catch the illegal all-zero state.
- **Results & Issues:** The LFSR correctly traversed 255 unique states before repeating and never entered the all-zero state.

1.8 Edge Detector and Debouncer

- **Design Approach:** Built a two-stage synchronizer (sync <= {sync[0], noisy_in}) to prevent metastability, feeding into a 4-bit counter-based debouncer. The debouncer waits for 16 consecutive identical samples before registering the state change. The edge detector applies a 1-cycle delay to find in & ~in_d.
- **Verification Steps:** Injected rapid, artificial toggles (noisy_in) mimicking switch bounce, let the signal settle, and monitored the output pulse count.
- **Results & Issues:** Verified successful debouncing. Despite the noisy input toggling 5 times, the edge detector fired only a single, clean one-cycle pulse once the debouncer counter saturated.